Algorithm Complexity Analysis

Step Counting & Asymptotic Analysis

Example 1 — Simple Sum Loop

ALGORITHM

```
int Sum(int n)
{
    int x = 0;
    for(int i = 0; i <= n; i++)
    {
        x = x + i;
    }
    return x;
}
```

STEP-BY-STEP COUNTING

- **int x = 0;**
  Executions / Steps: 1 time
- **for(i = 0; i <= n; i++)**
  Executions / Steps: (n + 1) + 1 times
- **x = x + i;**
  Executions / Steps: n + 1 times
- **return x;**
  Executions / Steps: 1 time

Total Step Formula: T(n) = 1 + (n + 2) + (n + 1) + 1 = 2n + 5

Asymptotic Complexity: O(n) [Linear Time]

Example 2 — Constant Loop Execution (n > 1)

ALGORITHM

```
// Suppose n > 1

i = n;
while(i > 1)
{
    i = i % 2;
    x = x + 1;
}
```

STEP-BY-STEP COUNTING

- **i = n;**
  Executions / Steps: 1 time
- **while(i > 1)**
  Executions / Steps: 2 times (entry test & termination)
- **i = i % 2;**
  Executions / Steps: 1 time (immediately sets i to 0 or 1)
- **x = x + 1;**
  Executions / Steps: 1 time

Total Step Formula: T(n) = 1 + 2 + 1 + 1 = 5

Asymptotic Complexity: O(1) [Constant Time]

Example 3 — Offset Loop Index

ALGORITHM

```
float sum(int n)
{
    int temp = 0;
    for(int i = 3; i < n; i++)
    {
        temp += 10;
    }
    return temp;
}
```

STEP-BY-STEP COUNTING

- **int temp = 0;**
  Executions / Steps: 1 time
- **for(i = 3; i < n; i++)**
  Executions / Steps: (n - 3) + 1 = n - 2 times
- **temp += 10;**
  Executions / Steps: n - 3 times
- **return temp;**
  Executions / Steps: 1 time

Total Step Formula: T(n) = 1 + (n - 2) + (n - 3) + 1 = 2n - 3

Asymptotic Complexity: O(n) [Linear Time]

Example 4 — Nested Loops

ALGORITHM

```
for(int i = 1; i <= n; i++)
{
    for(int j = 3; j < n; j++)
    {
        x = x + 1;
    }
}
```

STEP-BY-STEP COUNTING

- **Outer for(i = 1; i <= n; i++)**
  Executions / Steps: n + 1 times
- **Inner for(j = 3; j < n; j++)**
  Executions / Steps: n × ((n - 3) + 1) = n(n - 2) times
- **x = x + 1;**
  Executions / Steps: n × (n - 3) times

Total Step Formula: T(n) = (n + 1) + n(n - 2) + n(n - 3) = $2n^2 - 4n + 1$

Asymptotic Complexity: $O(n^2)$ [Quadratic Time]

Example 5 — Nested Loops

ALGORITHM

```
int x = 0;
for(int i = 0; i <= n; i++)
{
    for(int j = 2; j < n - 1; j++)
    {
        x = x + 1;
    }
    scanf("%d", &x);
}
return x;
```

STEP-BY-STEP COUNTING

- **int x = 0;**
  Executions / Steps: 1 time
- **Outer for(i = 0; i <= n; i++)**
  Executions / Steps: (n + 1) + 1 = n + 2 times
- **Inner for(j = 2; j < n - 1; j++)**
  Executions / Steps: (n + 1) × (n - 3 + 1) = (n + 1)(n - 2)
- **x = x + 1;**
  Executions / Steps: (n + 1) × (n - 3) times
- **scanf("%d", &x);**
  Executions / Steps: n + 1 times
- **return x;**
  Executions / Steps: 1 time

Total Step Formula: $T(n) = 2n^2 - n$

Asymptotic Complexity: $O(n^2)$ [Quadratic Time]

Example 6 — Linear Search without Early Return

ALGORITHM

```
int find(int key, int A[])
{
    int loc = 0;
    int i = 1;
    while(i <= n)
    {
        if(A[i] == key)
            loc = i;
        i++;
    }
    return loc;
}
```

STEP-BY-STEP COUNTING

- **loc = 0; i = 1;**
  Executions / Steps: 1 + 1 = 2 times
- **while(i <= n)**
  Executions / Steps: n + 1 times
- **if(A[i] == key)**
  Executions / Steps: n times
- **loc = i;**
  Executions / Steps: n times (worst-case all matches)
- **i++;**
  Executions / Steps: n times
- **return loc;**
  Executions / Steps: 1 time

Total Step Formula: T(n) = 2 + (n + 1) + n + n + n + 1 = 4n + 4

Asymptotic Complexity: O(n) [Linear Time]

Example 7 — Linear Search with Early Return

ALGORITHM

```
int find(int key, int A[])
{
    int loc = 0;
    int i = 1;
    while(i <= n)
    {
        if(A[i] == key)
            return loc;
        i++;
    }
    return loc;
}
```

STEP-BY-STEP COUNTING

- **loc = 0; i = 1;**
  Executions / Steps: 1 + 1 = 2 times
- **while(i <= n) [worst case]**
  Executions / Steps: n + 1 times
- **if(A[i] == key)**
  Executions / Steps: n times
- **i++;**
  Executions / Steps: n times
- **return loc; [at loop end]**
  Executions / Steps: 1 time

Total Step Formula: T(n) = 1 + 1 + (n + 1) + n + n + 1 = 3n + 4

Asymptotic Complexity: O(n) [Linear Time (Worst Case)]

Example 8 — Dependent Inner Loop (Summation)

ALGORITHM

```
for i = 0 to n - 1 do
{
    s = X[0];
    for j = 1 to i do
    {
        s = s + X[j];
    }
    A[i] = s / (i + 1);
}
return A;
```

STEP-BY-STEP COUNTING

- **Outer loop header (i = 0 to n-1)**
  Executions / Steps: n + 1 times
- **s = X[0]; and A[i] = s / (i + 1)**
  Executions / Steps: n + n = 2n times
- **Inner loop header (j = 1 to i)**
  Executions / Steps: 1 + 2 + 3 + ... + n = n(n+1)/2 times
- **s = s + X[j];**
  Executions / Steps: 0 + 1 + 2 + ... + (n-1) = n(n-1)/2 times
- **return A;**
  Executions / Steps: 1 time

Total Step Formula: $T(n) = (n + 1) + 2n + n(n+1)/2 + n(n-1)/2 + 1 = n^2 + 3n + 2$

Asymptotic Complexity: $O(n^2)$ [Quadratic Time]

Example 9 — Two Independent Variable Loop Bounds

ALGORITHM

```
total = 0;
for(i = 0; i <= n; i++)
{
    k = m;
    while(k > 1)
    {
        k = k - 1;
        total = total + 1;
    }
}
```

STEP-BY-STEP COUNTING

- **total = 0;**
  Executions / Steps: 1 time
- **Outer for(i = 0; i <= n; i++)**
  Executions / Steps: (n + 1) + 1 = n + 2 times
- **k = m; (inside outer loop)**
  Executions / Steps: n + 1 times
- **Inner while(k > 1)**
  Executions / Steps: (n + 1) × m times
- **k = k - 1; & total = total + 1;**
  Executions / Steps: (n + 1) × (m - 1) times each

Total Step Formula: T(n) = total = (n + 1)(m - 1)

Asymptotic Complexity: O(n · m) [Bivariate Complexity]

The Asymptotic Lexicon

Symbol	Name	Meaning
Θ	Tight Bound	The exact rate of growth. Bounded simultaneously from above and below.
O	Upper Bound	The worst-case scenario. The maximum limit of execution.
Ω	Lower Bound	The best-case scenario. The structural minimum of execution.
o	Strict Upper Bound	The function grows strictly slower than the bound.
ω	Strict Lower Bound	The function grows strictly faster than the bound.

Big-O Notation: Defining the Upper Bound

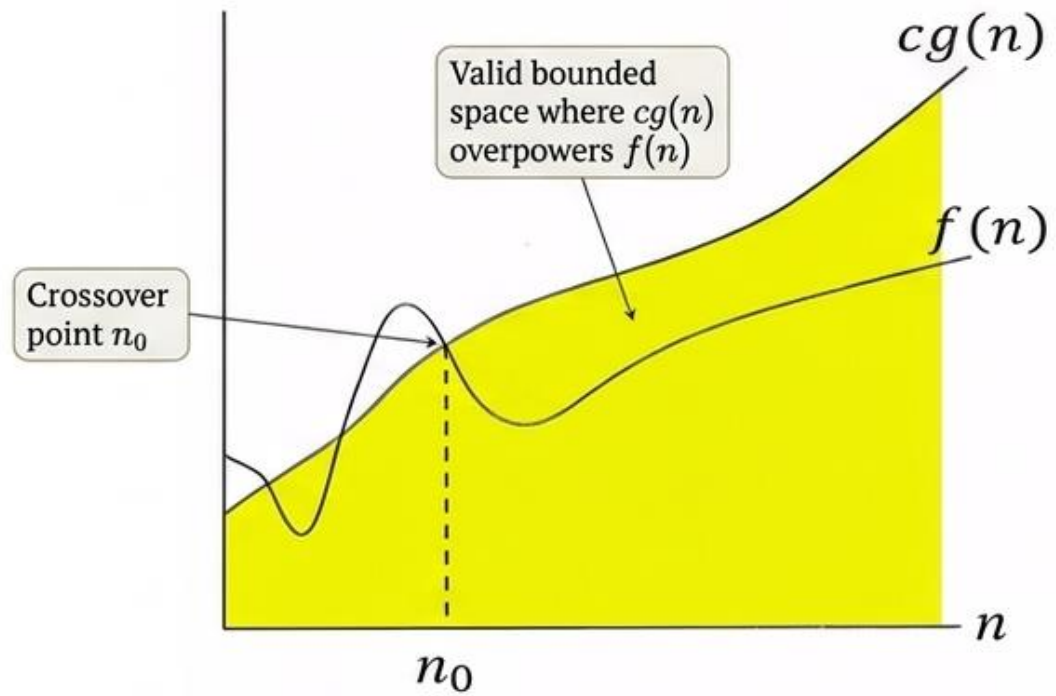
Proposition:

$$f(n) \in O(g(n))$$

Condition:

If there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0$$



Proving Big-O Boundaries

Goal: Show $n^2 + 10n \in O(n^2)$

Step 1: Set up the inequality:

$$n^2 + 10n \leq cn^2$$

Step 2: Choose constant $c = 2$:

$$n^2 + 10n \leq 2n^2$$

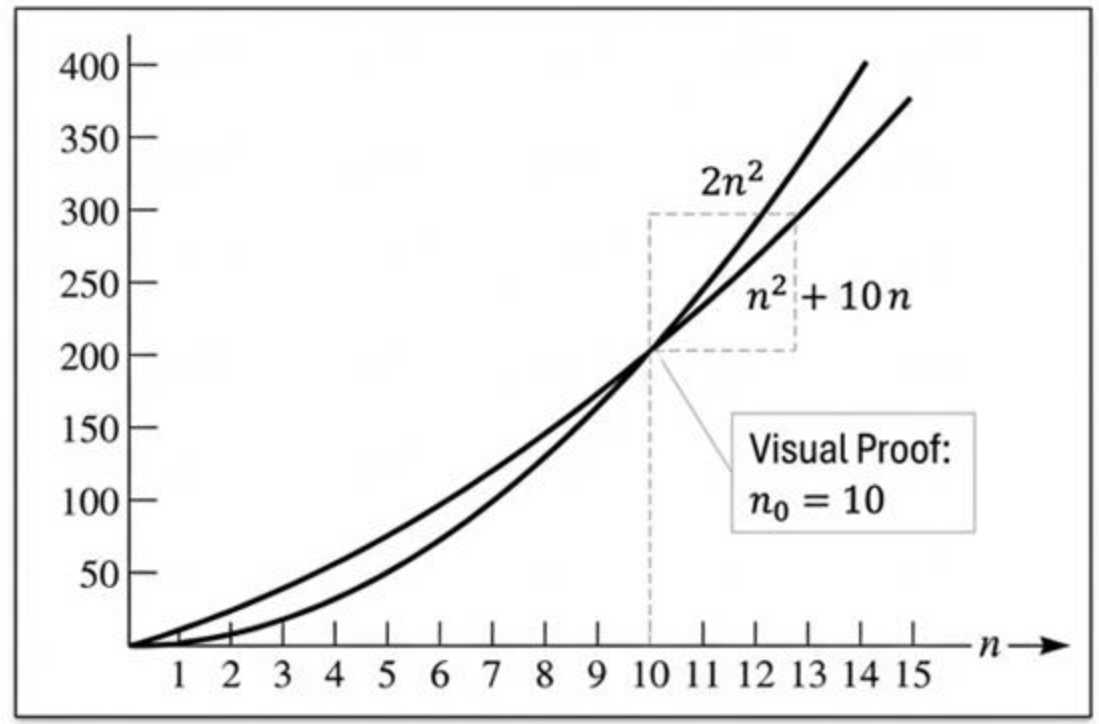
Step 3: Subtract n^2 from both sides:

$$10n \leq n^2$$

Step 4: Divide by n:

$$10 \leq n$$

Conclusion: The bound holds true for $c = 2$ when $n_0 \geq 10$.



Big-Ω Notation: Defining the Lower Bound

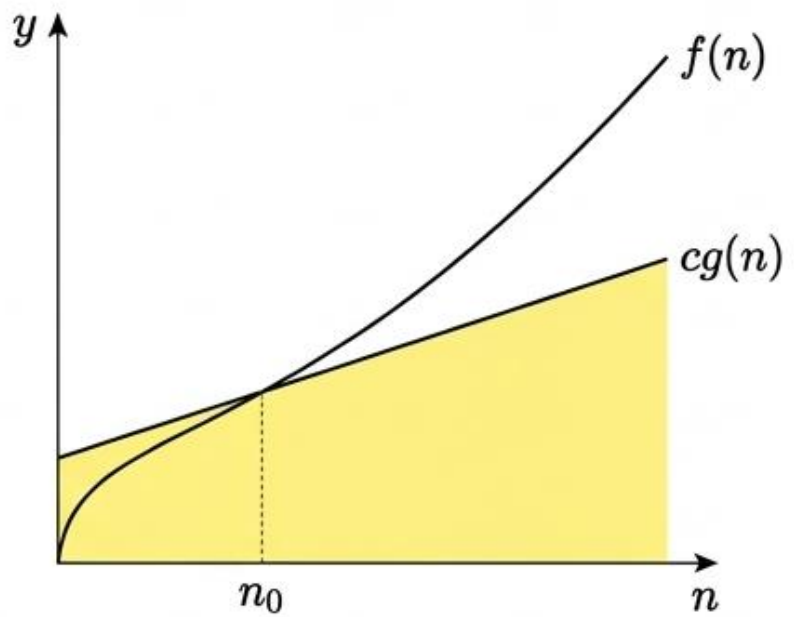
Proposition:

$$f(n) \in \Omega(g(n))$$

Condition:

If there exist positive constants c and n_0 such that:

$$0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0$$



The function $f(n)$ never drops below the structural minimum threshold $cg(n)$ after the crossover point n_0.

Big-Θ Notation: The Tight Bound

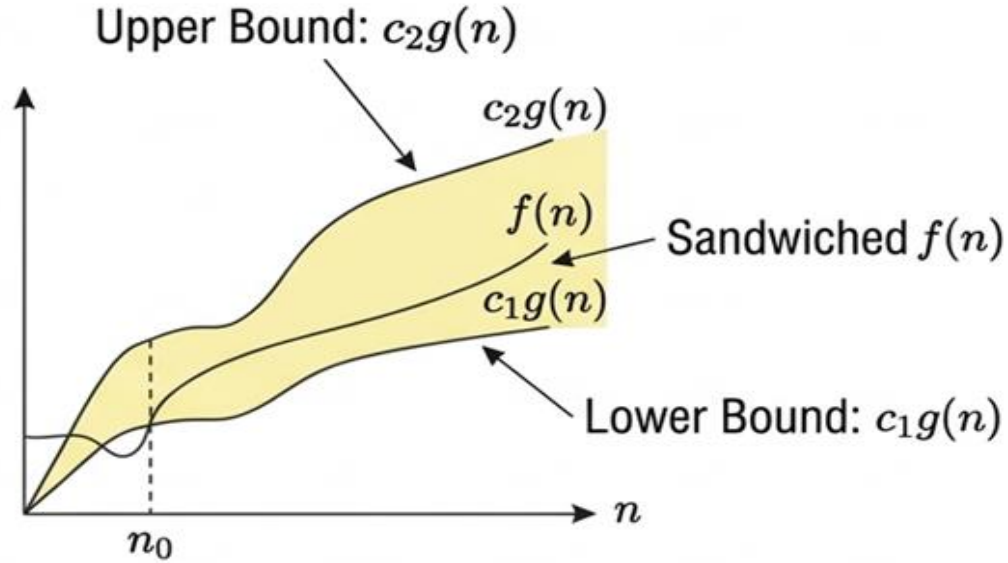
Proposition: $f(n) \in \Theta(g(n))$

Condition:

If there exist positive constants c_1, c_2, and n_0 such that:

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0$$

A tight bound implies the algorithm is bounded simultaneously from above (O) and below (Ω) by the exact same core function.



Proving Big-Θ Tight Bounds

The Proposition: Show $(1/2)n^2 - 3n \in \Theta(n^2)$

Target inequality: $c_1 n^2 \leq (1/2)n^2 - 3n \leq c_2 n^2$

↓

Deriving c_2:

$(\frac{1}{2})n^2 - 3n \leq c_2 n^2$

Let $c_2 = \frac{1}{2}$

Result: $-3n \leq 0 \Rightarrow n \geq 0$

Deriving c_1:

$c_1 n^2 \leq (1/2)n^2 - 3n$

Let $c_1 = \frac{1}{4}$.

Result: $\frac{1}{4})n^2 \leq (\frac{1}{2})n^2 - 3n$

$(\frac{1}{4})n^2 - 3n \geq 0 \Rightarrow n \geq 12$

↓

Conclusion:

The tight bound holds for constants $c_1 = 1/4$, $c_2 = 1/2$ starting at crossover $n_0 = 12$.

Strict Bounds: Little-o and Little-ω Limits

Little-o Notation

$$\lim_{n\to\infty}\left[\frac{f(n)}{g(n)}\right] = 0 \Rightarrow f(n) \in o(g(n))$$

Explanation: $g(n)$ grows strictly faster than $f(n)$. (Analogous to $<$)

Little-ω Notation

$$\lim_{n\to\infty}\left[\frac{f(n)}{g(n)}\right] = \infty \Rightarrow f(n) \in \omega(g(n))$$

Explanation: $g(n)$ grows strictly slower than $f(n)$. (Analogous to $>$)

Application Example

Question: Is $\frac{n^2}{5} \in \omega(n^3)$?

Limit Test:

$$\lim_{n\to\infty}\left[\frac{n^2/5}{n^3}\right] = \lim_{n\to\infty}\left[\frac{1}{5n}\right] = 0$$

Conclusion: No. The limit evaluates to 0, which means $\frac{n^2}{5} \in o(n^3)$.

Relational Properties of Growth Functions

Reflexivity	$f(n) \in O(f(n)), f(n) \in \Omega(f(n)), f(n) \in \Theta(f(n))$	(Equivalent to standard mathematical equality: $x = x$)
Transitivity	If $f(n) \in \Theta(g(n))$ and $g(n) \in \Theta(h(n))$ $\Rightarrow f(n) \in \Theta(h(n))$	(Applies universally across all 5 asymptotic notations)
Symmetry	$f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$	(Equivalent to: $x = y \Leftrightarrow y = x$)
Transpose Symmetry	$f(n) \in O(g(n)) \Leftrightarrow g(n) \in \Omega(f(n))$ Strict Transpose Symmetry: $f(n) \in o(g(n)) \Leftrightarrow g(n) \in \omega(f(n))$	(Equivalent to: $x \leq y \Leftrightarrow y \geq x$)

The Hierarchy of Algorithmic Complexity

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$$

$O(1)$ — Constant Time

$O(\log n)$ — Logarithmic Time

$O(n)$ — Linear Time

$O(n \log n)$ — Linearithmic / Log-Linear Time

$O(n^2)$ — Quadratic Time

$O(n^c)$ — Polynomial Time

$O(c^n)$ — Exponential Time